

Reviewer Pack — Frobenius Trace Defect of Clause-Indicator Polynomial Lower...

Entry b1916c940fcb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 17:25:42 UTC

Frobenius Trace Defect of Clause-Indicator Polynomial Lower-Bounds $\text{ACC}^0[2]$ Size

Entry ID: b1916c940fcb **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 17:25:42 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic of polynomial residues over $F_2[x]/(x^p - x)$: the Frobenius-trace defect $\text{tr}_p(P) := \dim_{F_2} (P - P^p \bmod (x^p - x))$ computed by direct evaluation, in the Lucas/Kummer combinatorial-trace tradition (rarely applied to circuit complexity) **Field B** (complexity object): $\text{ACC}^0[2]$ circuit size for Boolean functions presented as a CNF formula, the Williams-Murray target class for NEXP-ACC separations

Statement:

Let F be an unsatisfiable 3-CNF on n variables and let $P_F(x_1, \dots, x_n) := \sum \text{over clauses } C \text{ of } \prod_{l \in C} (1 - l)$ be its clause-indicator polynomial reduced mod 2. Define the Frobenius-trace defect at prime $p=3$ as $d_3(F) := |\{a \in F_3^n : P_F(a)^3 \neq P_F(a) \pmod{3}\}|$. Then the minimum $\text{ACC}^0[2]$ circuit size $S(F)$ satisfies $\log_2 S(F) \geq c \cdot d_3(F) / 3^{\lceil n/2 \rceil}$ for an absolute constant $c \geq 1/4$ across all unsatisfiable 3-CNFs with $n \leq 20$.

Rationale (proposer's reasoning):

The Williams-Murray algorithmic approach exploits that ACC circuits of size s admit succinct polynomial representations whose Frobenius-orbit structure mod small primes governs satisfiability counting; a non-vanishing trace defect mod 3 is exactly the obstruction that prevents collapse of P_F into a small-degree symmetric form and therefore lower-bounds any $\text{ACC}^0[2]$ simulation. Computing $d_3(F)$ by direct enumeration over F_3^n is feasible for $n \leq 20$ ($\sim 3^{20} \approx 3.5e9$ — restrict to $n \leq 12$ in tests), and $S(F)$ is upper-bounded by enumerating small $\{\text{AND}, \text{OR}, \text{NOT}, \text{MOD}2\}$ circuits, giving a falsifier-friendly inequality. The bridge is rare because Frobenius-trace combinatorics on clause polynomials has not, to our knowledge, been tied to ACC simulation size.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 93da3ff4beefb2a9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥ 200 random UNSAT 3-CNF instances spanning $n \in \{6, 8, 10, 12\}$ (≥ 50 per n , distinct seeds), compute $d_3(F)$ and $S_{\hat{}}(F)$ by gate-BFS. SUPPORT iff $\geq 95\%$ satisfy $\log_2(S_{\hat{}}) \geq 0.25 \cdot d_3^{(n/2)}$ with positive slack and per- n mean slack non-decreasing in n . FALSIFY iff any single instance violates the inequality.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.85	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	UNCERTAIN	0.78	HITS	SAFE
KARP_LIPTON	SAFE	0.80	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - ACC0 circuit lower bounds polynomial method F₃ CNF - Frobenius trace polynomial circuit complexity ACC modulo - clause indicator polynomial 3-CNF lower bound ACC[2] arithmetization

Top relevant hits considered: - [<http://arxiv.org/abs/2504.19966v3>] Quantum circuit lower bounds in the magic hierarchy - [<http://arxiv.org/abs/2304.02770v2>] Tight Correlation Bounds for Circuits Between AC0 and TC0 - [<http://arxiv.org/abs/2311.04204v3>] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [<http://arxiv.org/abs/2603.27466v1>] English translation of Frobenius' and Stickelberger's "On the theory of elliptic functions" - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1503.03164v2>] On the Frobenius complexity of determinantal rings - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/quant-ph/0305179v3>] Polynomial Degree and Lower Bounds in Quantum Complexity: Collision and Element Distinctness with Small Range

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def generate_unsat_3cnf(n):
    clauses = []
    for _ in range(2 * n):
        literals = [random.choice([1, -1]) * (i + 1) for i in range(n)]
        if sum(literals) != 0:
            clauses.append(tuple(sorted(literals)))
    return clauses

def evaluate_polynomial(poly, assignment):
    result = 1
```

```

for clause in poly:
    product = 1
    for literal in clause:
        var = abs(literal)
        sign = -1 if literal < 0 else 1
        if assignment[var] == 0:
            product *= (1 - sign)
        elif assignment[var] == 1:
            product *= sign
        else:
            raise ValueError("Invalid assignment value")
    result += product
return result % 2

def frobenius_trace_defect(poly, n):
    d_3 = 0
    for a in range(3**n):
        assignment = [(a // (3**(i-1))) % 3 for i in range(1, n+1)]
        if evaluate_polynomial(poly, assignment) ** 3 != evaluate_polynomial(poly, assignment):
            d_3 += 1
    return d_3

def build_circuit(n, depth):
    if depth == 0:
        return [random.choice([0, 1])]
    inputs = build_circuit(n, depth - 1)
    gate = random.choice(['AND', 'OR', 'NOT'])
    if gate == 'NOT':
        return [gate + '(' + str(inputs[0]) + ')']
    else:
        return [gate + '(' + str(inputs[0]) + ', ' + str(inputs[1]) + ')']

def evaluate_circuit(circuit, assignment):
    stack = []
    for token in circuit:
        if token.startswith('AND') or token.startswith('OR'):
            op2 = stack.pop()
            op1 = stack.pop()
            if token.startswith('AND'):
                result = 1 if int(op1) == 1 and int(op2) == 1 else 0
            else:
                result = 1 if int(op1) == 1 or int(op2) == 1 else 0
        elif token.startswith('NOT'):
            op = stack.pop()
            result = 1 - int(op)
        else:
            result = assignment[int(token)]
        stack.append(result)
    return stack[0]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [6, 8, 10, 12]
    results = []

    for n in n_values:

```

```

unsat_3cnf = generate_unsat_3cnf(n)
poly = defaultdict(int)
for clause in unsat_3cnf:
    poly[clause] += 1

d_3 = frobenius_trace_defect(poly, n)

max_depth = int(0.5 * n) + 2
best_size = float('inf')
for depth in range(max_depth):
    circuit = build_circuit(n, depth)
    if evaluate_circuit(circuit, [random.choice([0, 1]) for _ in range(n)]) == 1:
        size = sum(1 for token in circuit if isinstance(token, str))
        if size < best_size:
            best_size = size

S_hat_F = 2 ** (best_size - 1)

slack = math.log2(S_hat_F) - (0.25 * d_3 / 3**(n/2))
results.append({
    "metric_name": "slack",
    "metric_value": slack,
    "instances_tested": 1,
    "conjecture_holds": slack >= 0,
    "counterexample": ""
})

mean_slack = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
return {
    "seed": seed,
    "mean_slack": mean_slack,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9450f5e2.py", line 123, in <module>
    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9450f5e2.py", line 88, in run_trial

```

```

d_3 = frobenius_trace_defect(poly, n)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9450f5e2.py", line 45, in frobenius_tra
if evaluate_polynomial(poly, assignment) ** 3 != evaluate_polynomial(poly, assignment):
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9450f5e2.py", line 32, in evaluate_poly
if assignment[var] == 0:
~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` before producing any `RESULT` line, so the pre-registered support condition could not be evaluated. Per rule 2, a crash mandates an `INCONCLUSIVE` verdict. | next: Fix the variable-indexing bug in `evaluate_polynomial` (likely an off-by-one between 1-indexed clause literals and 0-indexed assignment list) and rerun the full 200-instance protocol across $n \in \{6,8,10,12\}$.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	22330
2	preregistration	claude_max	opus	0	0	6662
3	novelty	claude_max	opus	0	0	4145
4	novelty	claude_max	opus	0	0	11002
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13400
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11305
7	judge	claude_max	opus	0	0	4857

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 73702 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/b1916c940fcb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b1916c940fcb.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b1916c940fcb.tar.gz` (if generated)